AI Engineering and Claude Architect · Part V, Certification

Certification, Mock Exam 2

60 Items in 120 Minutes

Scenarios: Customer Support Resolution Agent, Developer Productivity with Claude, Claude Code for Continuous Integration, Structured Data Extraction. Set a timer, work without notes, and mark your answers on paper.

Items	60, in 4 scenario sets of 15
Time	120 minutes
Domains	16, 11, 12, 12 and 9 items, rounded from the guide's weights
Status	Original practice items, written from the guide. Not real or leaked

Before You Start

Key idea

These are original practice items, not real or leaked exam questions, and the company names are invented. A score here doesn't predict your exam result.

Follow the real conditions: one sitting, no notes, flag and return. Where the guide and current products differ, the answers follow the guide, and an explanation may add a Build today note. The real exam's wording may be longer and harder, as candidates report.

Checkpoints: 28 minutes after Scenario 1, 56 after Scenario 2, 84 after Scenario 3 and 110 after Scenario 4.

Targets, which are my own and not official: 80% overall, and no domain under 70%.

Scenario: Customer Support Resolution Agent

Key idea

Bluepeak Telecom's support agent runs on the Claude Agent SDK, using MCP tools for `get_account`, `lookup_invoice`, `apply_credit` and `escalate_to_human`. It handles billing disputes and account issues. Policy allows credits of up to $150 and covers only billing errors made by Bluepeak.

Q1. A test transcript shows the loop returning to the customer immediately after Claude requested `lookup_invoice`, so the invoice was never used. What was most likely skipped?

A. Raising `max_tokens`, so that Claude could include the invoice details in its first response
B. Starting a new conversation that contains only the tool's result, to reduce the input tokens
C. Parsing the response's text for a phrase that says that the assistant has now completed its work
D. Running the requested tool, appending its result to the history, and sending another request

Q2. An engineer adds a 10-iteration cap and treats reaching it as success. Which criticism is right?

A. A cap is correct as the main stopping signal, as long as its value is higher than the average number of tools
B. A cap can be a safety limit, but as the main stop it can cut work off, and treating it as success hides that
C. A cap breaks tool results, because the API deletes earlier tool messages once a tenth call has been made
D. A cap is unnecessary in every case, because the API refuses long-running loops at about ten iterations

Q3. Which rule most clearly needs programmatic enforcement rather than prompt guidance?

A. `apply_credit` must never run before the account holder's identity has been verified
B. The agent should prefer plain language over technical terms when it's explaining bills
C. The agent should offer a next step at the end of each reply to the customer
D. Replies should open by acknowledging what the customer has said about their issue

Q4. Which items should a mid-process escalation contain for a human who can't see the conversation? **Select 2.**

A. The raw JSON of every tool call made so far
B. The customer ID, root cause and amount
C. The agent's recommended action
D. The model's internal scratch reasoning, in full
E. A link to the agent's private conversation store

Q5. A customer disputes two invoices and also wants a plan change. The agent investigated only the first invoice. What's the best design change?

A. Summarize the customer's message into one sentence first, and then resolve whatever that one sentence ends up saying
B. Decompose requests into distinct items, investigate each with shared context, then give a unified resolution
C. Add a rule that says customers may bring up only one issue per conversation with the support agent
D. Create a separate top-level agent for each invoice, plus one more that handles the plan change alone

Q6. A compliance team requires that credits above $150 can never be applied by the agent. What's the correct mechanism?

A. A confidence score from the model, with credits blocked if its score falls below some threshold
B. A monthly audit of credit logs, along with retraining of the prompts where any violations appear
C. A longer system prompt, with several few-shot examples of credits that are above the limit
D. A hook that intercepts `apply_credit` and redirects anything above the limit to human escalation

Q7. `lookup_invoice` returns numeric status codes (1, 2, 3), and the agent sometimes misreads them. Which hook and action fit?

A. PreToolUse, translating the codes in the outgoing request into words
B. PostToolUse, translating codes into words before the model reads the result
C. UserPromptSubmit, translating the codes inside the customer's message
D. A Stop hook, translating the codes in the transcript once the whole conversation is over

Q8. Reviewing a billing dispute takes unknown steps, depending on what's found: it may need one invoice, or it may need a year of them. Which approach fits?

A. A batch job that investigates each dispute overnight and replies the next day
B. Adaptive investigation, which creates subtasks based on what each step finds
C. One large prompt containing every invoice the customer has ever received
D. A fixed chain that always loads twelve invoices, then summarizes all of them

Q9. A support session is resumed the next day, but the account's balance changed overnight, so earlier tool results are stale. Which approach is best, by the guide?

A. Start a fresh session with a structured summary of what still holds, and re-fetch current data
B. Resume the old session, as tool results in a resumed session are refreshed automatically
C. Fork the old session, as a fork fetches newer data for each of its previous tool calls
D. Resume the old session, since stale results are a minor risk that the customer would notice

Q10. `lookup_invoice` and `search_invoices` overlap, and selection is unreliable. Which description change helps most?

A. Shorten both descriptions to a few words, so that Claude's input tokens are used efficiently
B. State for each one the expected input, what it returns, and when to use it rather than the other
C. Mention in both descriptions that the tool is very important and should be used with care
D. List the two tools in a different order in the array, and add a number to each tool's name

Q11. A customer has no invoices for last month, so the query succeeds with no rows. How should `lookup_invoice` report it?

A. As `isError: true`, category permission
B. As a timeout, so that the agent retries it
C. As a successful result with no matches
D. As `isError: true`, category transient

Q12. Which `apply_credit` failure should have `isRetryable: false`?

A. The billing service returned a temporary 503 response
B. The credit violates policy because the error wasn't Bluepeak's
C. The service reported a short-lived rate limit on the call, and asked for a brief wait
D. The connection timed out before any response came back

Q13. A new "billing specialist" subagent keeps calling plan-change tools that it was given "just in case". What's the guide's reasoning for removing them?

A. MCP servers charge for each tool that's made available to each subagent during a session
B. A subagent may use only tools that its coordinator hasn't already used during the session
C. Plan-change tools would be blocked by the API, unless the subagent is using a higher model
D. Agents with tools outside their specialization tend to misuse them, so scope tools to the role

Q14. `get_account` returns 45 fields, and only 5 matter for billing. After several lookups, context is dominated by irrelevant data. What's the best fix?

A. Trim the results to relevant fields before they accumulate in context
B. Add an instruction telling Claude to pay no attention to irrelevant fields
C. Move to a model that has a larger context window, and keep all fields
D. Summarize the whole conversation after each one of the tool calls

Q15. A customer asks Bluepeak to match a competitor's price. Policy covers only Bluepeak's own billing errors. What should the agent do?

A. Refuse, because anything not in policy is forbidden

B. Grant a small credit as a goodwill gesture

C. Escalate, because policy is silent on this request

D. Ask the customer how much the competitor charges, then decide

Scenario: Developer Productivity with Claude

Key idea

Forge Tools builds an internal assistant on the Agent SDK. It helps engineers explore a 2-million-line codebase, understand legacy systems, generate boilerplate and automate repetitive tasks. It uses Read, Write, Edit, Bash, Grep and Glob, plus MCP servers.

Q16. A subagent that writes boilerplate needs the conventions found by the exploration subagent, but it keeps ignoring them. What's the fix?

A. Increase the second subagent's `max_turns`, so it has enough turns to recall the conventions
B. Have both subagents share one scratch variable, which the SDK keeps synchronized for them
C. Put the conventions the first subagent found directly into the second subagent's prompt
D. Tell the second subagent to ask the first one, by naming it in the second one's system prompt

Q17. Two teams disagree about the coordinator's prompt. One wants 15 numbered steps. The other wants the goal and quality criteria. What does the guide favor, and why?

A. Numbered steps, because subagents can't use any judgment of their own in the Agent SDK
B. Neither of these, because coordinators shouldn't send any instructions to their subagents
C. Numbered steps, because they're the only way of making subagents run in parallel at all
D. Goals and quality criteria, because they let subagents adapt to what they find

Q18. An engineer asks to "add comprehensive tests to this legacy module". What's the best first move?

A. Write tests from the first file alphabetically, and work on through to the very last file in the module
B. Generate one test per method, for all the methods, and fix whichever fail
C. Map the structure, identify high-impact areas, then create a prioritized plan that adapts
D. Ask for a list of required tests before reading anything in the module

Q19. A nightly review checks style, then security, then documentation for each changed file, always in the same way. Which pattern fits?

A. A hub-and-spoke system with a dozen subagents
B. A single prompt covering all aspects of all files
C. A fixed sequential pipeline (prompt chaining)
D. Dynamic decomposition with a new plan each run

Q20. An engineer wants to continue yesterday's investigation, which they named `auth-trace`. What's the guide's command?

A. `claude --resume auth-trace`
B. `claude --print auth-trace`
C. `claude --compact auth-trace`
D. `claude --memory auth-trace`

Q21. After analyzing a service, an engineer wants to try two testing strategies from that same analysis. What's best?

A. `fork_session` once for each strategy, so that both start from the shared baseline
B. Start two fresh sessions, repeating all of the analysis again from nothing in each
C. Run both strategies one after another in the same session, and compare the results
D. Use `/compact` between the strategies, to make room for the second one to be tried

Q22. A coordinator delegates "explore the payments code" to one subagent, and "find its tests" to another. Why route results through the coordinator, instead of letting them talk directly?

A. Observability, consistent error handling and controlled information flow
B. Direct exchange would make each subagent's context too large to use
C. Subagents can't produce any text unless a coordinator is reading their output at that time
D. The Messages API allows no more than two model calls per session

Q23. The team wants a shared GitHub MCP server, with each person's token kept out of version control. What's right?

A. Store the token in `CLAUDE.md`, and refer to it from `.mcp.json`
B. Have each engineer configure it in `~/.claude.json` by hand
C. Configure it in `.mcp.json`, with the token written in plainly
D. Configure it in `.mcp.json`, using `${GITHUB_TOKEN}` expansion

Q24. An engineer is trying an experimental MCP server just for themselves. Where should it go?

A. `.mcp.json` in the repo
B. The root `CLAUDE.md`
C. `~/.claude.json`
D. `.claude/rules/mcp.md`

Q25. The agent uses Grep for code search, and ignores a more capable MCP semantic search tool, which is described as "Searches code". What's the most appropriate fix?

A. Expand the MCP tool's description, explaining its capabilities and what it returns
B. Remove Grep from all agents, so that only the MCP tool remains available for search
C. Rewrite the MCP server in another language, so that its results come back more quickly
D. Add a sentence to `CLAUDE.md` that asks engineers to request the MCP tool by name

Q26. Engineers' agents make many exploratory calls to learn which internal docs exist. What should the server add?

A. A tool for each individual document
B. A longer system prompt describing the docs
C. A content catalog exposed as an MCP resource
D. A retry policy for exploratory calls

Q27. An engineer needs to find every file that matches `**/*.spec.ts`. Which tool fits?

A. Glob
B. Read
C. Edit
D. Grep

Q28. Edit fails because the target line appears 14 times in a file. What's the guide's fallback?

A. Use Glob to locate the right occurrence
B. Run Edit again until it picks the correct line
C. Read the full file, then Write the modified version
D. Delete the file, and generate it again from memory

Q29. An old module re-exports `calculateFee` under two other names through wrapper files, and a search for `calculateFee` misses callers. What's the right approach?

A. Glob for files named `calculateFee`, and treat each match as one of the callers
B. Identify all exported names first, then Grep for each one across the codebase
C. Read every file in the repository, because a text search can't be relied on at all
D. Ask the model which callers are typical of a module of this kind in this language

Q30. An engineer asks the assistant to explain a large unfamiliar subsystem, and the main session soon fills with file contents. Which feature is designed for this?

A. The batch API, which processes the discovery output overnight
B. A larger `max_tokens`, which leaves more room for the output
C. The Explore subagent, which isolates discovery and returns summaries
D. A forced `tool_choice`, which keeps responses shorter and quicker

Scenario: Claude Code for Continuous Integration

Key idea

Cobalt Systems runs Claude Code in CI. It reviews pull requests, posts inline comments and generates tests. Developers have begun to ignore the comments, as many are wrong or repeated.

Q31. A CI job hangs until its timeout whenever it runs Claude Code. What's the most likely fix?

A. Increase the pipeline's overall timeout
B. Add a `CLAUDE.md` to the pipeline folder
C. Run with `--resume`, using the last session
D. Run with `-p`, for non-interactive mode

Q32. The pipeline must post each finding as an inline comment, automatically. Which flags should it use?

A. `-p` with free-form text, and a regex to parse it
B. `--resume`, with a prompt asking for neat formatting
C. `--fork-session`, with one fork for each finding
D. `-p` with `--output-format json` and `--json-schema`

Q33. Each review run re-reports issues that it reported on earlier commits, including unresolved ones already under discussion. What's the best change?

A. Run reviews only on the final commit before merging, and skip all others
B. Give it prior findings, and ask it to report only new or still-unaddressed issues
C. Tell the model to be less repetitive, and to limit itself to five comments
D. Delete all previous comments before each run, so that each review starts on a clean page for developers

Q34. Generated tests duplicate existing ones and test trivial getters. Which changes address the causes? **Select 2.**

A. Provide existing test files in context
B. Document testing standards, valuable test criteria and fixtures in `CLAUDE.md`
C. Set a target for the number of generated tests
D. Add "write good tests" to the prompt
E. Switch this job to the batch API

Q35. Claude generates a feature in a session, then the same session reviews it, and the review finds little. Why, and what's better?

A. Reviewing is poorly suited to models, so have humans review all generated code alone
B. The model is tired by that point, so wait a while, and then review in the same session
C. Reviews need a larger context, so ask the same session to compact and then review
D. The session retains its reasoning, so use an independent instance without that context

Q36. Review criteria for `**/migrations/**` should apply only when those files are in play, wherever they are in the repo. Which fits best?

A. A `.claude/rules/` file with a `paths` glob
B. A setting in the user's home directory
C. A section in the root `CLAUDE.md`
D. A `CLAUDE.md` file in each migrations folder

Q37. Reviews in CI don't follow standards that local sessions follow. The standards are in each developer's `~/.claude/CLAUDE.md`. What should be done?

A. Move them to project-level configuration, which CI receives with the repository
B. Add the phrase "follow team standards" to the prompt the pipeline sends to Claude
C. Copy a developer's home directory onto the CI runner's image during each build
D. Add a CI step that asks a developer to approve each review before it's posted

Q38. A review skill must not be able to modify files. Which frontmatter is intended for that, in the guide's terms?

A. `context: fork`, naming the read tools
B. `paths`, limiting the writable files
C. `allowed-tools`, listing only read tools
D. `argument-hint`, listing only read tools

Q39. A developer wants to switch the whole pipeline from one review architecture to another, across many services, with several viable options. How should they begin?

A. With a batch job that tries each option overnight
B. In plan mode, investigating before committing to changes
C. With a fresh `CLAUDE.md` describing the target design, and no further exploration
D. With direct execution, because it's only a pipeline

Q40. Claude keeps mishandling null values in a migration script's edge cases. Which feedback is most effective?

A. Restarting with the same prompt again
B. A specific test case with example input and expected output
C. "Please handle null values better the next time you edit this migration script"
D. A longer explanation of what null means

Q41. You've written the test suite first for a parser that Claude will write. How should iteration work?

A. Share test failures with Claude, and iterate until they pass
B. Merge the first version, then write up any failures that production finds
C. Hide the test results, so that Claude writes its code without bias
D. Ask Claude to change tests whenever a failure proves hard to fix

Q42. "Check that comments are accurate" causes many false positives. Which rewrite follows the guide?

A. "Report all possible comment issues, and include your level of confidence in each"
B. "Flag comments only when their claimed behavior contradicts the code's actual behavior"
C. "Be very careful, and report only the comment problems that you're sure about"
D. "Use your best judgment as an experienced developer when you're checking comments in this codebase"

Q43. Security findings are accurate, but style findings are mostly wrong, and developers now ignore everything. What's the best immediate action?

A. Add a confidence score to each finding, and leave all categories turned on as they are
B. Keep posting everything, and ask developers to be more patient during the process
C. Disable all categories until the prompts have all been redesigned from the beginning
D. Temporarily disable the high false-positive style category while improving its prompt

Q44. A 22-file PR gets shallow feedback, with contradictions between comments on different files. What's the best design?

A. One pass, with a request for greater thoroughness and more detailed reasoning
B. A larger context window for the same pass, with all 22 files included at once
C. Per-file passes for local issues, plus a separate cross-file integration pass
D. A rule that blocks all pull requests with more than ten changed files from merging

Q45. Which uses of model confidence does the guide support? **Select 2.**

A. Using raw confidence to discard findings without any calibration
B. Having the model report confidence alongside each finding
C. Escalating customer cases on model confidence alone
D. Treating high confidence as proof that a finding is correct
E. Using calibrated confidence to route review attention

Scenario: Structured Data Extraction

Key idea

Veritas Health extracts fields from clinical trial summaries and insurance forms into a registry. Accuracy matters more than speed. Layouts vary, and fields are often absent. Downstream systems require strict JSON.

Q46. Extraction misses citations when they appear inline, but it catches them in bibliographies. What's the most effective change?

A. Add an enum value "citation" to all the fields in the output schema
B. Run the same prompt twice, then merge the two sets of results
C. Add few-shot examples of correct extraction from each of the two structures
D. Add a longer instruction saying that citations can be found in various places

Q47. Informal measurements such as "about a cup" lead to invented precise values. What does the guide suggest?

A. Few-shot examples showing how to handle informal measurements correctly
B. Asking for a confidence score, then using the value anyway
C. Removing all documents with informal language from the input
D. A stricter numeric type, so that only precise values pass the schema

Q48. What can a tool schema with strict validation not prevent?

A. A string given where a number is required
B. A value placed in the wrong but correctly typed field
C. A trailing comma that makes the JSON invalid
D. A missing field that the schema has marked required

Q49. You must run `extract_metadata` before any enrichment tools. What should the first request do?

A. Remove the enrichment tools from the API for all later requests
B. Use `tool_choice: "any"`, with a prompt that asks for the right tool first
C. Force that tool, with `tool_choice: {"type": "tool", "name": "extract_metadata"}`
D. Use `tool_choice: "auto"`, while listing `extract_metadata` first in the tools

Q50. A field records adverse-event severity. The documents sometimes use unexpected terms, or are genuinely unclear. Which schema suits this? **Select 2.**

A. A numeric score the model assigns from 1 to 10
B. A free-text string with no constraints at all
C. Enum including "unclear"
D. Enum including "other", with a detail string
E. Enum with only "mild", "moderate" and "severe", required

Q51. Which failure is most likely to be fixed by a retry with error feedback?

A. The date is in the document, but in the wrong format
B. The enrollment figure isn't anywhere in the document
C. The sponsor name appears only in a separate document
D. The file provided is a blank scanned page

Q52. Developers keep dismissing findings from a form-check pipeline. How can you analyze which constructs cause false positives?

A. Lower the number of findings at random until dismissals begin to decline
B. Stop recording dismissals, which only encourage developers' negative opinions
C. Ask developers to write a paragraph explaining each of their dismissals
D. Add a `detected_pattern` field to findings, then analyze dismissals by pattern

Q53. Before sending 60,000 forms through a batch, what should you do to reduce wasted resubmissions?

A. Remove `custom_id`, to make the requests smaller and simpler to send
B. Submit them all synchronously first, to check them, and then as a batch
C. Submit them all at once, and fix the prompt according to the failures
D. Refine the prompt on a sample, to maximize the first-pass success rate

Q54. Twenty section summaries are aggregated into one input, and findings from the middle sections keep being omitted. What's the guide's mitigation?

A. Increase `max_tokens`, so that the model's reply has enough room to cover more of the sections
B. Remove the headers, so that the model reads the sections as continuous text
C. Shuffle the order of the sections randomly, on each and every request
D. Put a key findings summary first, and organize the details under explicit headers

Q55. A downstream agent with a small context budget gets long analyses from an upstream agent. What's the best change?

A. Have the upstream agent return structured key facts, citations and relevance scores
B. Have the downstream agent truncate its input at a fixed number of characters
C. Have the downstream agent ask the upstream agent to repeat itself more slowly
D. Have the upstream agent write longer explanations of its reasoning, so that its conclusions are clearer

Q56. A registry lookup returns no match for a sponsor. Later, you find the registry was down. What's the design flaw?

A. The extraction should have stopped altogether, on any registry query
B. The model should have been asked whether the registry was available
C. The lookup should always have returned the closest available sponsor
D. An access failure was reported the same way as a valid empty result

Q57. You plan to automate extractions above a confidence threshold. How should the threshold be set?

A. From the average confidence that the model reports across all the documents in production
B. From field-level confidence, calibrated on a labeled validation set
C. From a round number such as 0.9, which is a common choice
D. From what leadership would like the automation rate to be

Q58. Overall accuracy is high, but your report groups all documents together. What's the main risk?

A. Field-level confidence scores stop being output
B. Stratified sampling becomes impossible in that situation
C. Poor accuracy on a document type or field can stay hidden
D. The aggregate is too low to be statistically meaningful

Q59. Two credible trial summaries report different response rates. One is from 2023 and one from 2026. How should the system handle them? **Select 2.**

A. Keep both values, with attribution to their sources
B. Drop both, because conflict makes them unusable
C. Average the two, then cite both of them
D. Include publication dates, so a time difference isn't misread as a conflict
E. Choose the higher value, as it's the more useful

Q60. Summarized findings no longer say where each claim came from. What's the root-cause fix?

A. Ask the summarizer to use more formal, academic language for its claims
B. Require structured claim-source mappings that later steps must preserve
C. Use a larger model for the summarization step in the pipeline
D. Add a bibliography of all sources consulted to the end of each report

Answer Key and Score Sheet

Q	Ans	Q	Ans	Q	Ans	Q	Ans
1	D	2	B	3	A	4	B, C
5	B	6	D	7	B	8	B
9	A	10	B	11	C	12	B
13	D	14	A	15	C	16	C
17	D	18	C	19	C	20	A
21	A	22	A	23	D	24	C
25	A	26	C	27	A	28	C
29	B	30	C	31	D	32	D
33	B	34	A, B	35	D	36	A
37	A	38	C	39	B	40	B
41	A	42	B	43	D	44	C
45	B, E	46	C	47	A	48	B
49	C	50	C, D	51	A	52	D
53	D	54	D	55	A	56	D
57	B	58	C	59	A, D	60	B

Domain	Items	Your score	Target
1. Agentic Architecture and Orchestration	16		70%+
2. Tool Design and MCP Integration	11		70%+
3. Claude Code Configuration and Workflows	12		70%+
4. Prompt Engineering and Structured Output	12		70%+
5. Context Management and Reliability	9		70%+
Total	**60**		**80%+**

The guide doesn't publish how raw scores map to the scaled 720, so these targets are only study guidance.

Your error log: item, what you picked, why it was wrong

Explanations

1. D (TS 1.1). `stop_reason: "tool_use"` means Claude is waiting for results. They're appended so it can reason about the next step. Text parsing and fresh conversations are anti-patterns, and `max_tokens` is unrelated.

2. B (TS 1.1). The guide lists arbitrary caps as the primary stop among the anti-patterns. `stop_reason` is the primary signal. The other claims about API behavior are false.

3. A (TS 1.4). Financial ordering requires a deterministic guarantee. The rest concern tone and style, where prompts are appropriate and an occasional miss isn't a compliance failure.

4. B and C (TS 1.4). The guide lists customer details, root cause and recommended actions. Raw reasoning and full JSON bury those facts, and the human doesn't have access to the store.

5. B (TS 1.4). This is the guide's multi-concern pattern. A one-issue rule fails the customer, separate agents lose shared context, and a one-sentence summary drops concerns.

6. D (TS 1.5). "Never" means a deterministic guarantee, so use a hook. Prompts remain probabilistic, an audit only detects after the fact, and confidence isn't a policy check.

7. B (TS 1.5). The codes come back in the tool's result, so the hook has to act after the tool runs and before the model reads it.

8. B (TS 1.6). Open-ended investigation suits dynamic decomposition. A fixed chain and a large prompt waste context, and a batch job doesn't suit a live support conversation.

9. A (TS 1.7). When prior tool results are stale, a fresh session with an injected summary is more reliable. Resuming doesn't refresh anything, and neither does forking.

10. B (TS 2.1). Differentiation is what's missing. Emphasis, shortening and reordering don't tell Claude when to choose which.

11. C (TS 2.2). A valid empty result is a success. Reporting it as an error sends the agent into pointless recovery.

12. B (TS 2.2). A policy violation won't change on retry. The other three are transient.

13. D (TS 2.3). Scoped access is about misuse and selection reliability. The other statements aren't how tools or the API work.

14. A (TS 5.1). Trimming addresses the cause. Summaries risk losing details, ignoring doesn't free tokens, and a bigger window delays the problem and adds cost.

15. C (TS 5.2). A policy gap is an escalation trigger. Refusing and improvising both invent policy that the company hasn't set.

16. C (TS 1.3). Context must be passed explicitly. The SDK provides no such shared variable, more turns won't reveal what it never received, and routing goes via the coordinator.

17. D (TS 1.3). The guide specifies goals and quality criteria rather than procedures. The other options misstate how subagents work.

18. C (TS 1.6). The guide's example decomposes by mapping first and adapting as dependencies appear. The others ignore impact and discovery.

19. C (TS 1.6). Predictable multi-aspect work suits prompt chaining. Dynamic planning adds cost without benefit, a single prompt invites attention dilution, and a dozen subagents overbuild.

20. A (TS 1.7). `--resume <session-name>` continues a named session. The other flags don't do this.

21. A (TS 1.7). Forks create independent branches from a shared baseline. In a single session, the first strategy influences the second. Fresh sessions repeat the analysis. **Forks share the same filesystem.**

22. A (TS 1.2). These are the guide's reasons for hub-and-spoke. The others aren't true constraints.

23. D (TS 2.4). Project scope shares the config, and env expansion keeps the secret out. Plain tokens or tokens in `CLAUDE.md` get committed, and user scope stops it being shared.

24. C (TS 2.4). User-level configuration is for personal or experimental servers. The others affect the team or aren't server configuration.

25. A (TS 2.4). The guide's fix for preferring built-ins is a better description. Removing Grep loses a useful tool, and the other options don't address why Claude chose it.

26. C (TS 2.4). Resources give visibility into available data without exploratory tool calls. The others add load, or don't help discovery.

27. A (TS 2.5). Glob matches file paths by pattern. Grep searches contents.

28. C (TS 2.5). Read + Write is the guide's fallback. Build today: a longer anchor or `replace_all` may also work. Glob finds files and not lines, and regenerating from memory risks data loss.

29. B (TS 2.5). The guide traces wrappers by first finding exported names. Reading everything wastes context, file names don't show callers, and "typical" isn't evidence.

30. C (TS 3.4). Explore preserves main-session context during verbose discovery. The others don't address where the file contents go.

31. D (TS 3.6). Without `-p`, Claude Code waits for interactive input. The others don't change that.

32. D (TS 3.6). JSON output with a schema gives machine-parseable findings. Regex parsing of prose is brittle, and the others don't produce structure.

33. B (TS 3.6). With prior findings in context, it can avoid duplicates. Deleting comments loses history, later-only runs lose early feedback, and a cap doesn't deduplicate.

34. A and B (TS 3.6). Existing tests prevent duplicates, and documented standards shape value. A numeric target rewards more low-value tests, batch changes timing only, and "good" isn't a criterion.

35. D (TS 3.6). Session isolation is the guide's point. Compacting keeps the same reasoning thread, there's no such fatigue, and dropping AI review isn't what's asked.

36. A (TS 3.3). Path rules load conditionally, regardless of directory. Multiple directory files are harder to maintain, root content always loads, and user-level isn't shared with CI.

37. A (TS 3.1). CI gets what's in the repository. Home directory copying is fragile and personal, an approval doesn't supply standards, and the phrase names nothing specific.

38. C (TS 3.2). The guide says `allowed-tools` restricts tool access during a skill. Build today: current docs say it pre-approves tools, so use deny rules to truly block them.

39. B (TS 3.4). Large, multi-file, multi-option changes suit plan mode. The others commit too soon, or skip investigation.

40. B (TS 3.5). Specific cases give Claude something concrete to satisfy. The others carry no new information.

41. A (TS 3.5). The guide's test-driven iteration shares failures to guide improvement. Hiding them removes the signal, and changing tests undermines them.

42. B (TS 4.1). Explicit criteria define what to report. "Be careful" and confidence-based filtering are the guide's examples of what doesn't help.

43. D (TS 4.1). Disabling the noisy category restores trust and keeps valuable findings. Disabling all loses security value, patience doesn't fix trust, and confidence scores don't fix wrong categories.

44. C (TS 4.6). Multi-pass reduces attention dilution and contradictions. Larger windows don't fix attention, and blocking PRs avoids the question.

45. B and E (TS 4.6). The guide supports verification passes and calibrated routing. Uncalibrated filtering and treating confidence as truth are what it warns against. Self-reported confidence is also an unreliable escalation signal.

46. C (TS 4.2). Few-shot examples demonstrate handling of varied document structures. The rest don't teach the variation.

47. A (TS 4.2). The guide says few-shot examples reduce hallucination for cases such as informal measurements. A stricter type pressures the model to invent, and removing documents isn't a fix.

48. B (TS 4.3). Schemas remove syntax and shape errors, not semantic ones. A trial's enrollment number in the dosage field can still be a valid number.

49. C (TS 4.3). Forcing guarantees the first step, and follow-up turns handle enrichment. `auto` and `any` allow others. Build today: forced selection is rejected on some current models.

50. C and D (TS 4.3). Escape values let the model be accurate. A closed required enum forces a wrong fit, free text loses structure, and a numeric score invents precision the source lacks.

51. A (TS 4.4). Format and structure errors respond to feedback. Absent data doesn't, and the same goes for a document that was never provided.

52. D (TS 4.4). `detected_pattern` enables systematic analysis. Paragraphs are hard to aggregate, and the others remove the signal.

53. D (TS 4.5). Refining on a sample avoids costly iteration at scale. `custom_id` is how you identify failures, so removing it would hurt.

54. D (TS 5.1). This is the "lost in the middle" effect. Placement and headers mitigate it. Shuffling only moves the loss, and `max_tokens` limits output rather than attention.

55. A (TS 5.1). Structured output carries what's needed at lower cost. Fixed truncation loses content arbitrarily.

56. D (TS 5.3). Access failures and valid empty results need to be distinguished. Guessing, stopping everything and asking the model are all worse.

57. B (TS 5.5). Thresholds need calibration against labeled data. Uncalibrated numbers don't say what accuracy they give.

58. C (TS 5.5). Aggregates can mask poor segments. The other statements don't follow from how the report is grouped.

59. A and D (TS 5.6). The guide says to annotate conflicts with attribution and require dates. Choosing, averaging or dropping loses what the reader needs.

60. B (TS 5.6). Claim-source mappings keep the link. A bibliography doesn't say which claim came from which source.